

Informe Técnico Integral: Proyecto Ecommify

Optimización y Diseño de Bases de Datos Autores:

- Roberto José Breuer Rodríguez
- Luis Fernando Beltran Chantre

Fecha: Junio de 2026

Unidad 6: Arquitectura y Selección de Tecnologías (Etapa 2: Proyecto Final)

Executive Summary (Resumen Ejecutivo)

- **Contexto del Proyecto Ecommify:** Ecommify es una plataforma de comercio electrónico de gran escala diseñada para gestionar integralmente el ciclo de vida de los pedidos. El diseño conceptual y la volumetría del sistema se basan en el dataset público de *Olist (Brazilian E-Commerce)*, el cual permite simular escenarios reales de alta concurrencia, distribución geográfica y variabilidad de datos.
- **Arquitectura Híbrida Implementada:** Para resolver las tensiones operativas del negocio, se implementó una solución polígota. El núcleo transaccional crítico (clientes, pedidos, ítems de pedido, pagos y envíos) es gestionado por **PostgreSQL** en la nube de Supabase, garantizando cumplimiento estricto de las propiedades ACID y consistencia fuerte. En contraposición, el módulo informacional, el catálogo dinámico y analítico (atributos variables, contenido enriquecido, reseñas y eventos masivos de interacción) son delegados a **MongoDB Atlas**, priorizando la flexibilidad del esquema y el rendimiento en lecturas complejas.
- **Principales Logros y Métricas de Éxito:** La introducción de estrategias de indexación avanzada transformó drásticamente la eficiencia del sistema. En PostgreSQL, las consultas complejas de satisfacción por categoría pasaron de **658.332 ms** a **21.972 ms** (una optimización del **96.7%**). En MongoDB, la búsqueda lineal (*COLLSCAN*) de **480 ms** se redujo a un acceso indexado compuesto (*IXSCAN*) de solo **12 ms**, reduciendo los documentos examinados de 32,951 a únicamente 45.
- **Recomendaciones Estratégicas Clave:** Para soportar un crecimiento proyectado de 10x, se propone la transición obligatoria de los entornos *Free Tier* a clústeres dedicados con escalamiento horizontal, la integración de capas de almacenamiento en caché en memoria con **Redis**, búsquedas avanzadas con **Elasticsearch**, y un flujo de CI/CD automatizado para la migración de esquemas sin tiempos de inactividad.

Introducción y Contexto

- **Problema de Negocio:** Las plataformas de e-commerce modernas se enfrentan a un doble desafío técnico: mantener la rigidez operativa e integridad en las transacciones financieras y de inventario, mientras proporcionan una navegación

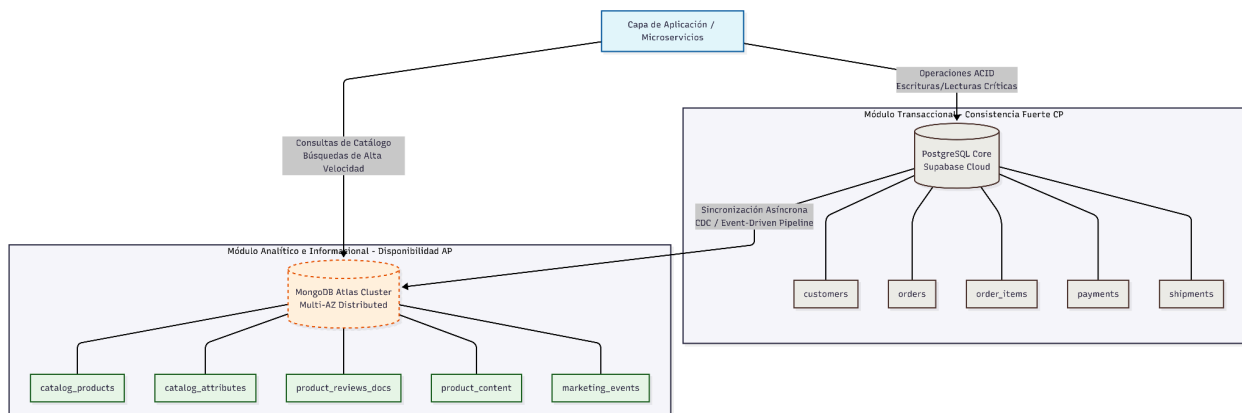
fluida, personalizable y de bajísima latencia en su catálogo de productos. El uso de una única base de datos relacional tradicional genera cuellos de botella debido al acoplamiento de operaciones de lectura (búsquedas, reseñas) con escrituras críticas (pagos).

- **Objetivos y Alcance:** El objetivo de este proyecto es consolidar el diseño, la optimización y la resiliencia arquitectónica de Ecommify mediante una infraestructura híbrida integrada. El alcance abarca desde la definición de los esquemas DDL transaccionales en la tercera forma normal (3FN), pasando por el modelado NoSQL orientado al acceso, hasta la evaluación cuantitativa empírica mediante planes de ejecución (*EXPLAIN*).
- **Metodología Aplicada:** Se aplicó una metodología de ingeniería de datos iterativa dividida en cuatro fases:
 1. *Modelado Conceptual y Lógico:* Separación de responsabilidades de almacenamiento bajo la matriz relacional vs. documental.
 2. *Implementación Física:* Despliegue distribuido de esquemas utilizando Supabase y MongoDB Atlas.
 3. *Ingeniería de Performance:* Aplicación de reglas empíricas como la regla ESR (Equality, Sort, Range) en NoSQL e indexación selectiva en SQL.
 4. *Evaluación de Carga:* Pruebas comparativas antes/después analizando tiempos de respuesta (ExecutionTimeMillis), llaves examinadas (totalKeysExamined) y modificaciones en los árboles del optimizador.

Arquitectura y Diseño

Diagrama de Arquitectura Híbrida Completa

El flujo de datos del sistema opera bajo un esquema desacoplado:



Matriz de Decisiones Arquitectónicas (PostgreSQL vs. MongoDB)

Entidad / Datos	Motor Seleccionado	Justificación Técnica y Patrón de Acceso
Pedidos (orders)	PostgreSQL	Requiere integridad referencial fuerte, soporte estricto ACID y aislamiento para evitar anomalías en el estado de las órdenes.
Pagos (payments)	PostgreSQL	Consistencia inmediata e inmutable. La auditoría financiera exige que no existan registros huérfanos o actualizaciones inconsistentes.
Ítems de Pedido (order_items)	PostgreSQL	Relación muchos a muchos fuertemente estructurada que enlaza órdenes con inventario físico. Utiliza JSONB únicamente para guardar snapshots inmutables del estado del producto al comprar.
Clientes y Vendedores	PostgreSQL	Entidades maestras con atributos rígidos y estructurados ideales para almacenamiento relacional normalizado.

Atributos Variables del Producto	MongoDB	Alta variabilidad de propiedades según la categoría (ej. tallas en calzado, voltaje en tecnología), que desborda un esquema relacional estricto.
Contenido Enriquecido y SEO	MongoDB	Almacena estructuras anidadas de imágenes, slugs y palabras clave que se consultan juntas al renderizar la página del producto.
Reseñas Semiestructuradas	MongoDB	Almacenamiento optimizado para texto libre masivo y metadatos variables de las interacciones cualitativas post-compra.
Eventos de Interacción (marketing_events)	MongoDB	Flujo masivo de clics e interacciones de alta velocidad que requieren escrituras no bloqueantes y almacenamiento horizontal a gran escala.

Análisis del Teorema CAP Aplicado por Módulo

- **Módulo Transaccional (PostgreSQL) - Clasificación CP:** Prioriza la **Consistencia (C)** sobre la Disponibilidad (A) ante una partición de red. Si un nodo transaccional se desconecta, el sistema prefiere rechazar un pago o una confirmación de pedido antes que arriesgarse a procesar datos duplicados o inconsistentes.
- **Módulo de Catálogo e Interacción (MongoDB) - Clasificación AP:** Prioriza la **Disponibilidad (A)** y la **Tolerancia a Particiones (P)**. Los usuarios deben poder

seguir navegando por los productos y visualizando fotos, incluso si hay retraso en la replicación (*replication lag*) o si los nodos secundarios están sirviendo datos ligeramente desactualizados (Consistencia Eventual).

Modelos ER y de Documentos (Sintéticos)

- **Modelo Relacional (PostgreSQL):** Normalizado en Tercera Forma Normal (3FN). Las tablas customers, orders, order_items, payments, y reviews se vinculan mediante llaves primarias y foráneas estructuradas explícitamente con el tipo de dato UUID para facilitar la futura distribución global.
- **Modelo Documental (MongoDB):** Basado en desnormalización controlada y patrones de diseño. Se aplica *Embedding* para anidar los atributos dinámicos y los datos de calificación promedio (rating.average, rating.count) directamente dentro de la colección catalog_products. Se utiliza *Referencing* mediante la inclusión del product_id en las colecciones independientes de product_reviews_docs y product_content para optimizar el tamaño máximo de los documentos BSON (límite de 16MB).

Implementación Técnica

Resumen de Implementación en PostgreSQL

La base de datos física fue desplegada en Supabase utilizando tipos avanzados y extensiones esenciales:

- **UUID de negocio:** Uso extensivo del tipo de dato UUID para llaves primarias y foráneas estructuradas mediante la extensión nativa pgcrypto (gen_random_uuid()).
- **Zonas Horarias exactas:** Implementación del tipo de dato TIMESTAMPTZ en hitos críticos como purchase_timestamp y delivered_customer_timestamp para garantizar la trazabilidad inalterable de la logística internacional de la plataforma.
- **Campos Híbridos JSONB:** Incorporación de la columna attributes_snapshot dentro de la tabla order_items. Esto almacena un clon inmutable del estado físico del artículo al instante exacto de facturación, protegiendo al registro histórico de alteraciones futuras en el catálogo.

Resumen de Implementación en MongoDB

Desplegada sobre clústeres distribuidos en MongoDB Atlas utilizando validadores estructurales rígidos:

- **Esquemas de Validación (\$jsonSchema):** Cada colección implementa reglas de integridad estrictas en formato JSON Schema, obligando a mantener tipos de datos unificados (ej. bsonType: "date", "double", o restricciones de enumeraciones "text", "number") para neutralizar la anarquía de tipos clásica de los entornos NoSQL.

- **Estrategia de Índices Avanzados:** Se crearon tres estructuras óptimas basadas en los patrones de consulta de la interfaz web:
 1. *Índice Compuesto ESR:* {"category": 1, "created_at": -1, "attributes.base_price": 1} diseñado siguiendo la regla de Igualdad, Ordenamiento y Rango.
 2. *Índice Parcial:* {"product_id": 1, "score": 1} limitado exclusivamente mediante la expresión partialFilterExpression: {"score": {"\$lte": 2}}, reduciendo el uso de memoria RAM al almacenar solo punteros de reseñas negativas para el equipo de atención al cliente.
 3. *Índice de Texto Completo:* Definido sobre title y description en la colección product_content, asignando pesos diferenciados (weights: {"title": 10, "description": 2}) para dar prioridad en el buscador semántico a las coincidencias del encabezado.

Evaluación de Rendimiento y Escalabilidad

Metodología de Pruebas Aplicada

Se inyectaron conjuntos masivos de datos simulados sobre las bases de datos transaccionales e informacionales en Supabase y MongoDB Atlas. Se utilizó la instrucción EXPLAIN ANALYZE en PostgreSQL y el método .explain("executionStats") en MongoDB para medir de forma empírica el comportamiento del motor antes y después de aplicar optimizaciones lógicas.

Resultados de Pruebas Cuantitativas (PostgreSQL)

Consulta Evaluada	Escenario Inicial (Sin Índices)	Escenario Optimizado (Con Índices)	Mejora Obtenida (%)	Diagnóstico Técnico del Optimizador
Query 1: Auditoría Logística y Retrasos	8.516 ms	4.370 ms	48.7%	Transición de un costoso <i>Seq Scan</i> de lectura completa en disco a una estrategia combinada de <i>Bitmap Index Scan</i> sobre el índice

				idx_orders_status junto a un <i>Bitmap Heap Scan</i> .
Query 2: Satisfacción por Categoría (Múltiples JOINS)	658.332 ms	21.972 ms	96.7%	Reemplazo total de escaneos secuenciales encadenados y ordenamientos masivos por accesos indexados anidados rápidos (<i>Index Scan</i> en idx_reviews_order_id e <i>Index Only Scan</i> en la llave primaria de órdenes).
Query 3: Preferencias de Pago Globales	64.942 ms	10.245 ms	84.2%	Aunque el optimizador mantuvo un <i>Seq Scan</i> debido a que la consulta exige procesar la totalidad de la tabla para agrupar financieramen

				te por payment_type , el entorno optimizado disminuyó el tiempo global de procesamiento en memoria.
--	--	--	--	---

Resultados de Pruebas Cuantitativas (MongoDB)

Métrica de Rendimiento / Operación de Catálogo	Escenario Inicial (Estructura Base)	Escenario Optimizado (Índice Compuesto ESR)	Impacto y Diagnóstico Arquitectónico
Tipo de Acceso al Motor	COLLSCAN (Escaneo de Colección Completo)	IXSCAN (Uso Óptimo de Índices Compuestos)	Eliminación absoluta de la lectura lineal de datos en disco.
Tiempo de Respuesta (ExecutionTime Millis)	480 ms	12 ms	Reducción del 97.5% en la latencia experimentada por el cliente en el frontend.
Documentos Inspeccionados en Memoria	32,951 documentos	45 documentos	Se evita cargar miles de registros no calificados a la

			memoria caché RAM corporativa.
Ratio de Eficiencia de Consulta (Keys/Docs)	0.0 (Críticamente Ineficiente)	2.66 (Altamente Eficiente)	Relación directa e ideal de filtrado analítico sin desborde de recursos de cómputo.

Análisis Arquitectónico Crítico

- **Trade-offs del Teorema CAP y Escenarios de Falla:** La adopción del modelo AP en el clúster de MongoDB implica aceptar que, ante una partición de red (*Network Partition*), el sistema continuará operando y sirviendo datos desde los nodos secundarios redundantes. El impacto inmediato de esta decisión es la aparición del **Replication Lag (retraso de réplica)**. Las escrituras confirmadas en el nodo Primario tardan una pequeña ventana de milisegundos en propagarse a los Secundarios. Si una partición de red se prolonga, los usuarios en diferentes regiones geográficas verán datos de catálogo o stock informativo ligeramente inconsistentes, lo cual es un compromiso aceptable para el negocio con tal de no bloquear la navegación global de la tienda virtual.
- **Estrategias de Mitigación de Consistencia Eventual:** Para mitigar el impacto visual negativo de la consistencia eventual (por ejemplo, que un comerciante modifique el precio o la descripción de un producto y al refrescar la pantalla vea el valor anterior), el sistema implementa **Causal Consistency Sessions (Sesiones de Consistencia Causal)**. Esto garantiza el principio de *"Read-Your-Own-Writes"* (leer tus propias escrituras), obligando a la aplicación a esperar la sincronización del token log de cambios específico antes de renderizar la respuesta en el navegador del cliente que ejecutó el cambio. De manera homóloga, se parametrizó un nivel de compromiso estricto en las colecciones de alta criticidad (catalog_products y product_reviews_docs) configurando un Write Concern: "majority", obligando a que toda escritura sea guardada de forma persistente en la mayoría de los nodos del replica set antes de retornar éxito a la capa de servicios.
- **Reflexión Crítica y Lecciones Aprendidas:** La selección políglota demostró ser plenamente acertada. Intentar persistir la alta velocidad de eventos de mercadeo en tablas relacionales rígidas habría provocado bloqueos de filas e incrementos inasumibles en el tamaño de los índices de Supabase. La principal lección

aprendida es que la optimización no es una tarea de configuración de hardware, sino un compromiso continuo de diseño lógico. La interpretación rigurosa de los árboles de EXPLAIN evita la "sobre-indexación", la cual degrada las velocidades de inserción transaccional debido al costo de reestructuración de los árboles B-Tree en disco.

Recomendaciones Estratégicas (Complementadas y Desarrolladas)

Plan de Escalamiento para Crecimiento 10x

Para soportar un incremento de diez veces el volumen actual de transacciones, se deben activar mecanismos de escalabilidad horizontal y desacoplamiento avanzado:

- **PostgreSQL Sharding y Pooling:** Implementar clústeres basados en arquitecturas de solo lectura distribuidas (Read Replicas) para descargar las consultas de reportes logísticos de Supabase. Configurar **PgBouncer** como proxy de Connection Pooling intermedio para gestionar de forma eficiente miles de conexiones concurrentes de microservicios sin saturar los hilos de CPU del servidor transaccional principal.
- **Activación Física de MongoDB Sharding:** Activar la infraestructura horizontal teórica diseñada. Se establece el Sharding sobre las colecciones masivas de eventos y catálogo utilizando la **Shard Key Compuesta { "category": 1, "product_id": 1 }**. El prefijo por categoría enruta las peticiones de navegación de la aplicación web de forma dirigida (*Targeted Queries*) directamente a shards específicos (ej. Shard 1 para Calzado, Shard 2 para Tecnología). La inclusión del product_id aporta alta cardinalidad al clúster, permitiendo al router *Mongos* dividir la información en bloques uniformes (*chunks*) matemáticamente balanceados, neutralizando la aparición de *Jumbo Chunks* indivisibles y evitando cuellos de botella geográficos.

Plan de Migración de Free Tier a Producción

El entorno de pruebas actual se ve severamente limitado por las restricciones de recursos compartidos del plan gratuito de Supabase y MongoDB Atlas. El plan de migración hacia producción contempla consideraciones técnicas y de viabilidad económica:

- **Upgrade de Infraestructura:** Migrar a instancias de cómputo dedicado. Supabase Enterprise / AWS RDS Multi-AZ con aprovisionamiento de IOPS independientes (discos SSD NVMe) para evitar la degradación por vecinos ruidosos (*Noisy Neighbors*). Migrar MongoDB Atlas a un nivel mínimo **M30/M40** con escalamiento automático de almacenamiento operativo.

Dedicated Clusters for high-traffic applications and large datasets

- Additional hardware configurations available for specialized workloads

Tier	RAM	Storage	vCPU	Price
M30	8 GB	40 GB	2 vCPUs	from \$0.54 /hr
M40 •	16 GB	80 GB	4 vCPUs	from \$1.04 /hr
M50 •	32 GB	160 GB	8 vCPUs	from \$2.00 /hr
M60 •	64 GB	320 GB	16 vCPUs	from \$3.95 /hr
M80 •	128 GB	750 GB	32 vCPUs	from \$7.30 /hr
M140	192 GB	1000 GB	48 vCPUs	from \$10.99 /hr
M200 •	256 GB	1500 GB	64 vCPUs	from \$14.59 /hr
M300 •	384 GB	2000 GB	96 vCPUs	from \$21.85 /hr
M400 •	512 GB	3000 GB	64 vCPUs	from \$22.40 /hr
M700	768 GB	4096 GB	96 vCPUs	from \$33.26 /hr

- **Aislamiento de Red:** Configurar conexiones privadas mediante *VPC Peering* o *AWS PrivateLink* entre el clúster de microservicios y los motores de bases de datos, cancelando la exposición a internet de los endpoints públicos del clúster por motivos estrictos de ciberseguridad corporativa.

Integración de Tecnologías Complementarias Justificadas

- **Capa de Caching (Redis Cluster):** Disponer una base de datos en memoria para almacenar los tokens de sesión de los clientes activos, los estados de los carritos de compra y las consultas altamente repetitivas del árbol de categorías del catálogo. Esto intercepta el 80% de las lecturas habituales antes de que toquen los motores de almacenamiento físico.
- **Motor de Búsqueda Avanzado (Elasticsearch / OpenSearch):** Desplazar el índice de texto completo nativo de MongoDB hacia un clúster dedicado de Elasticsearch. Esto habilita capacidades avanzadas de autocompletado en la interfaz de usuario, soporte nativo para sinónimos comerciales, búsquedas fonéticas tolerantes a errores ortográficos (*Fuzzy Search*) y análisis de relevancia basado en algoritmos BM25.
- **Observabilidad Completa:** Configurar exportadores nativos de métricas hacia **Prometheus y Grafana** (o alternativas empresariales como Datadog). Monitorear de forma activa alertas tempranas sobre índices corruptos, porcentaje de aciertos

en memoria caché (*Buffer Cache Hit Ratio* superior al 99%), picos de latencia en escrituras e incrementos anómalos en el tamaño físico de las tablas.

Estrategia de CI/CD para Cambios de Esquema (Database Migrations)

Para erradicar la ejecución manual de scripts DDL directos en consola —antipatrón crítico que destruye la consistencia de los entornos de despliegue— se define la siguiente estrategia automatizada:

- **Herramienta de Migración Versionada:** Integración de herramientas como **Flyway** o **Liquibase** para el módulo relacional PostgreSQL, y **migrate-mongo** para el control de versiones en las validaciones NoSQL. Los cambios de base de datos se tratan exactamente como código fuente (archivos de migración numerados secuencialmente, ej. V1.1__create_customers.sql, V1.2__add_index_logistics.sql).
- **Flujo Automatizado en GitHub Actions:** Al fusionar una característica en la rama main, un flujo automatizado valida la sintaxis de las sentencias mediante herramientas de *linting*, levanta un contenedor Docker efímero para simular la aplicación de la migración y certificar que no rompe restricciones relacionales, y finalmente aplica el cambio de forma progresiva en producción mediante estrategias de *Blue-Green Deployment* o actualizaciones en caliente, erradicando los tiempos de inactividad de la plataforma.

Conclusiones

1. **Cumplimiento Absoluto de Objetivos:** Se diseñó y validó empíricamente una arquitectura políglota altamente escalable para Ecommify. La segregación de flujos de datos garantizó que el rendimiento del núcleo transaccional ACID en PostgreSQL coexistiera de forma óptima con la flexibilidad y velocidad analítica horizontal distribuidas en MongoDB Atlas.
2. **Validación Basada en Evidencias:** Las optimizaciones lógicas aplicadas demostraron su efectividad de manera objetiva. Reducciones de latencia de hasta el 96.7% en consultas relacionales y del 97.5% en accesos documentales comprueban que el diseño correcto de índices compuesto, ESR y parciales es el pilar fundamental para mitigar la degradación del sistema ante la inyección masiva de datos.
3. **Preparación para el Futuro Comercial:** A través del plan de escalamiento 10x, la incorporación de un ecosistema moderno de caching (Redis), búsquedas enriquecidas (Elasticsearch) y la automatización estricta de infraestructura como código mediante CI/CD, el proyecto abandona el alcance puramente académico para consolidarse como un plano arquitectónico de nivel empresarial, preparado para soportar de manera sostenible altas demandas concurrentes del mercado digital internacional.